Techfest 2026-27

TrueFoundry Forge-a-thon

Objective

Every engineer has used an AI coding tool, and most have an opinion about where it falls short: it loses track of the file you care about, it rewrites a function you did not ask it to touch, it confidently edits a symbol that does not exist. This competition asks you to build the thing you have opinions about, on the same foundation that production agents run on.

Your mission is to ship a coding agent as a Visual Studio Code extension whose agent loop runs on **TrueForge**, TrueFoundry's open-source agent harness. Given a real repository, it must read the code, plan a change, edit several files, and present those edits for review inside the editor. You choose the AI model it runs on. TrueForge handles the loop, so your energy goes where the difficulty really is: deciding what the model sees, and making the editor experience safe and fast.

Challenge Question

How would you build an editor-native coding agent on TrueForge that finds the right code in a repository larger than any model's context window, makes a multi-file change the user never had to spell out file by file, shows every change for review before it touches disk, and keeps working when the model behind it fails?

Track-specific specs/deliverables

What You Must Build

Five capabilities are mandatory. A submission missing one can still be scored, but it will score poorly on capability coverage. This is the floor, not the contest; the contest is in how well you build them.

1. **Inline edit.** The user selects a region of code, presses a keybinding, and types an instruction in plain language. Your extension returns a proposed diff rendered in the editor, which the user accepts or rejects. A chat reply the user must copy and paste does not count.
2. **Agent mode.** A panel where a single request drives a TrueForge agent through a tool loop (read file, search, write file, run a terminal command) until the task is done or the agent gives up. It must carry out a change spanning at least three files without the user naming them. Finding them is the agent's job.
3. **Context assembly.** Explicit @file and @symbol mentions, plus automatic retrieval of relevant code the user did not mention. This is the intellectual core of the problem. Once a repository no longer fits in a context window, what you put in and what you leave out decides whether the agent works at all. TrueForge's context-engineering features are there to be used, and using them well is scored.
4. **Diff review.** Changes are presented hunk by hunk with a visible preview, and the user accepts or rejects each one. No silent writes to disk. An applied change must be undoable.
5. **Streaming and cancellation.** Tokens appear as they arrive through TrueForge's streaming events. The user can interrupt a generation mid-flight and the editor never blocks. A cancelled run leaves the workspace consistent, with no half-applied edits.

Out of scope (not scored; time spent here is time lost): tab completion or fill-in-the-middle; training, fine-tuning or hosting your own model; a hosted backend, database or deployed service beyond TrueForge in local mode; user accounts or authentication; publishing to the VS Code Marketplace.

Bring Your Own Model

You choose the AI model and where it runs. Any of the following is acceptable, configured in TrueForge: a paid API from any provider, a provider's free tier, a model served through an AI gateway, or an open-source model running on your own machine behind an OpenAI-compatible endpoint. Your extension must work

with whatever model TrueForge is configured with; it must not depend on one provider.

Fairness. Points are awarded for behaviours your extension demonstrates, never for how clever the model's answers are. A team using a free or local model that demonstrates every behaviour scores exactly the same as a team using an expensive model. Judges score what the extension does on screen: whether a diff is rendered, whether files were discovered, whether a cancelled run left the workspace clean. The organisers may ask any shortlisted team to repeat a fixed task live with a model of the organisers' choice before the shortlist is final, so build for any model, not for one.

Declare it. Your write-up must state every model you used, where it ran, and roughly what it cost. Never commit an API key to your repository; ship an example configuration file instead.

Building on Open Source

You are encouraged to reuse open-source code for the editor side of the problem: the webview, diff rendering, keybindings and VS Code plumbing from projects such as Cline, Continue, Roo Code or opencode. Rebuilding a diff viewer from scratch is not a demonstration of skill. The agent loop, however, must be replaced with TrueForge; keeping an upstream project's own loop is an eligibility failure. We score your delta, not your repository:

- Declare your fork point in the README: the upstream repository URL and the exact commit you started from.
- Preserve the upstream licence and attribution. A licence violation is an eligibility failure.
- Describe what you changed and why, in the README or in your write-up. Judges read your git history from the declared fork commit forward.

Track-specific environment stack

START HERE: the TrueForge GitHub repository
https://github.com/truefoundry/trueforge

Every submission must run its agent loop on TrueForge. Star the repository, read the README, and run npx @truefoundry/trueforge@latest on your laptop before you write a line of extension code. Documentation: trueforge.dev · Quickstart: trueforge.dev/quickstart · Questions: TrueForge community Discord

Format of competitions & Evaluation criteria

Round 1

The Three Fixed Tasks

Every team demonstrates the same three tasks on the same repository, so that a judge watching the three-hundredth video is comparing like with like.

Repository: pallets/flask, release tag 3.1.3 (commit 22d92470). Clone it and check out the tag before recording. It is about 17,000 lines of Python across src and tests, larger than most context windows, which is the point.

Exact wording will not change. You may rehearse as often as you like.

Task	Exact instruction to give your agent	How to verify it on screen
A. Inline edit (single file)	Open src/flask/helpers.py, select the function get_debug_flag, and instruct: *"Treat the values 'off' and 'n' as false as well, case-insensitively, and update the docstring to list every value that is treated as false."* Review and accept	Run FLASK_DEBUG=off python -c "from flask.helpers import get_debug_flag; print(get_debug_flag())" and show False.

	the diff.	
B. Agent mode (three or more files, none named)	In the agent panel, with no files mentioned, instruct: *"Add a new built-in CLI command flask config that prints the loaded application's configuration as JSON, masking the values of any key containing SECRET, PASSWORD, TOKEN or KEY as "***". It must appear in flask --help, be covered by a test, and be documented alongside the other built-in commands."* Review the hunks and accept. Show the TrueForge session for this run.	Run pytest tests/test_cli.py -q and show it passing, then run the new command against any Flask app and show the masked JSON. The diff must touch at least three files.
C. Resilience	Configure TrueForge with an unreachable endpoint or an invalid key, then repeat Task A.	Show that the editor stays responsive, the user sees a clear message (or a second configured model takes over automatically), and git status shows no files changed.

Submission Guidelines

Three artefacts, all mandatory, all submitted through the Techfest Competition Portal as links before the Round 1 submission deadline. Links that are not public at the deadline are not scored; we will not request access.

Artefact	**Requirements**
Demo video	Maximum 4 minutes. Screen recording with the editor visible. Tasks A, B and C in that order, each followed by its on-screen verification, with the TrueForge session shown briefly during Task B. You may cut out model waiting time; do not cut between giving an instruction

	and the diff appearing. Narration or captions optional. Public link (YouTube unlisted or Google Drive "anyone with the link can view").
Write-up	PDF, 1 to 3 pages, with these eight headings in order: **1. Team** (members, college or employer, GitHub handles). **2. Starting point** (upstream repository and commit, or "from scratch"). **3. Architecture on TrueForge** (how the extension talks to TrueForge, which editor capabilities are exposed as MCP tools, which harness features you used and why). **4. How context is selected** (your retrieval strategy and its budget). **5. Model setup** (models used, where they ran, approximate cost, how a user switches models). **6. Scorecard** (a table with one row per judging criterion: the video timestamp where it is shown and a one-line claim). **7. Tools used** (including any AI coding assistants; this costs no points). **8. Known limitations** and what you would do with another month.
GitHub repository	Public before the deadline. README with setup steps that work from a clean clone, including how to start TrueForge locally and load your agent definition; the declared fork point if any; an example model configuration file. Upstream LICENSE preserved. No API keys or secrets committed.

Evaluation Criteria

Eligibility gate (pass or fail, checked first)

The agent loop must run on TrueForge (github.com/truefoundry/trueforge). Concretely:

- Your VS Code extension is a client of a TrueForge agent. It creates sessions and sends turns through the TrueForge HTTP API or the TypeScript SDK, and renders the streamed events in the editor.
- Your editor's capabilities (read file, search the workspace, propose a write, run a command) are exposed to the agent as MCP tools, so the agent, not the extension, decides what to read and change.
- Run TrueForge in local mode on your own machine (npx

@truefoundry/trueforge@latest). Nothing needs to be deployed. The sandbox feature is optional and not required for any fixed task.

- Your model is configured in TrueForge as a catalog provider or an OpenAI-compatible endpoint. Your extension must not call a model provider directly.
- Your demo video must briefly show the TrueForge session for Task B (turns and tool calls), and your repository must contain the agent definition you used.

Why this rule: a harness exists so that you do not spend your month rebuilding streaming, session state, tool plumbing and approvals. With those solved, the competition is about the two things that separate a good coding agent from a bad one: context assembly and a safe, fast editor experience.

Scoring: 100 points

Every criterion uses written anchors tied to fixed values. A judge selects the description that fits, not a number. Where a submission sits between two anchors, the lower one applies.

Criterion	Points	Anchors
A. Capability coverage (30)		
A1 Inline edit	10	**0** not present, or output the user must copy by hand · **4** returns an edit but no diff in the editor · **7** diff rendered, accept-all only · **10** diff rendered with per-hunk accept and reject
A2 Agent mode	10	**0** no tool loop · **4** edits a single file only · **7** edits several files, but the user had to name them · **10** discovers and edits three or more files unprompted (Task B)
A3 Streaming and	10	**0** editor blocks during generation · **4** streams, no cancellation · **7** cancellation works but can leave half-applied edits · **10** cancellation is immediate and the

cancellation		workspace stays consistent
B. Technical depth (45)		
B1 Context assembly	15	**0** whole files concatenated until the window fills · **5** explicit @-mentions only · **10** automatic retrieval, strategy stated in the write-up · **15** retrieval with a size budget, strategy stated, and visibly at work in Task B (the agent shows which files it chose and why)
B2 Diff safety	10	**0** writes to disk without showing the user · **5** full-file preview before write · **10** per-hunk review and a working undo of an applied change, shown on screen
B3 Resilience (Task C)	10	**0** hang, stack trace, or files changed · **4** clear error message, workspace unchanged · **7** retry with backoff, then a clear error, workspace unchanged · **10** automatic fallback to a second configured model with no user action, workspace unchanged
B4 TrueForge integration depth	10	**4** sessions and turns only; editor actions handled outside the harness · **7** editor capabilities exposed as MCP tools, with TrueForge tool approval driving the diff review · **10** also makes meaningful use of at least one context-engineering feature (subagents, deferred tool loading, Code Mode, compaction) or skills, explained in the write-up and visible in the session
C. Presentation (25)		
C1 Video	10	**0** a fixed task missing or out of order · **5** all three present but outcomes not legible or not verified · **10** all three complete, legible, and visibly verified on screen
C2 Write-up	15	**0** absent or boilerplate · **5** describes features without design decisions · **10** states the architecture, context strategy, model setup and trade-offs · **15** also carries an

		accurate scorecard with timestamps and an honest limitations section

Process and shortlist

- Each submission is scored independently by two judges; the score is the mean. A spread of more than 15 points triggers a third judge.
- The **top 15 teams** by score are shortlisted for Round 2. Ties break on B, then A, then earlier submission time.
- Before the shortlist is final, each shortlisted team joins a 10-minute video call to walk through its code and repeat Task A live with a model chosen by the organisers. A team that cannot explain its own submission is replaced by the next team in rank.
- Round 1 scores are published to each team with the anchor selected for every criterion.

Round 2

The 15 shortlisted teams travel to Techfest, IIT Bombay, for Round 2 on 16 to 18 December 2026. Round 2 will involve building agents on TrueForge and, in addition, raising a Pull Request to the TrueForge repository (github.com/truefoundry/trueforge): a contribution to the harness itself, reviewed as part of your score. TrueFoundry will provide model credits to every finalist team for Round 2, so no team's result depends on the model it can afford. The Round 2 problem statement, format and judging criteria are shared with all shortlisted teams before the finale day, at the same time, so every finalist has equal time to prepare. Expect to operate what you build in front of judges, on material you have not seen before, and to answer questions on your design.

Social Media Contest (optional)

Build in public. This contest is optional and independent of the technical score; a team may win a Social Media Award prize.

- **Platforms:** LinkedIn, X (Twitter) and YouTube. Post on any or all three, one post per platform counts.
- **Content:** each post must show what you built and include your demo (a clip, a screen recording or the demo video link).

- **Hashtags:** every post must include **#TrueForge, #Forgeathon** and **#<YourTeamName>**. Tagging the official TrueFoundry and Techfest accounts is recommended.
- **Window:** posts published between 15 October and 18 December 2026.
- **Submission:** one link per platform, up to three links per team, submitted through a form that TrueFoundry will share by 18 December 2026. Only links submitted through the form are counted.
- **Announcement:** winners are announced in December after the final round concludes.

Evaluation Criteria

A single number per team = reactions on the LinkedIn post + impressions on the X post + views on the YouTube video, as shown on the platforms at the time of counting. Highest total wins first prize; the next two are runners-up; the next five receive special mentions. Artificial engagement (bought or bot-driven) disqualifies the team from the contest.

Team Specifications and Eligibility

- Eligibility: open to anyone aged 18 or over on the date of registration. Students from any college and working professionals are welcome. A valid government-issued or institution-issued photo ID is required.
- Team size: up to 5 members. Members may be from different colleges or organisations, and a team may mix students and professionals.
- Each participant may register only once and may be part of only one team.

Timeline and key dates

Event	**Deadline**
Registration opens	15 October 2026
Registration closes	15 November 2026
Round 1 submission deadline	20 November 2026, 23:59 IST
Round 1 Results	30 November 2026
Round 2 problem statement and model credits shared with finalists	Before the finale day; date announced with the shortlist

Round 2 (Finals at Techfest IIT Bombay)	16th-18th December
Social media links form shared by TrueFoundry	18th December
Social Media Awards announced	December 2026, after the final round
Prize money form deadline	31 December 2026

General rules of the competition

1. Teams must register online at techfest.org and will receive a Team ID for future reference.
2. The agent loop must run on TrueForge. Extensions that call a model provider directly or keep an upstream project's own agent loop are ineligible.
3. All code must be the team's own work, except for declared forks and ordinary dependencies. Open-source libraries and tools may be used with licence compliance and credit.
4. You may use any AI coding tool to build your coding agent. Declare it in the write-up; it costs no points.
5. Models used must be declared in the write-up and configured in TrueForge. Hard-coding one provider is not acceptable.
6. Do not use code from an employer or any private codebase. Use the pinned repository and your own work.
7. Submissions must be public under an open-source licence compatible with any upstream project used. By submitting, teams grant TrueFoundry and Techfest, IIT Bombay a non-exclusive, perpetual, royalty-free licence to use, modify and showcase the submission, and agree that registration details are shared with TrueFoundry subject to applicable law.
8. The organisers' and judges' decisions are final and binding. Techfest reserves the right to verify participant details.
9. Techfest, IIT Bombay, is not responsible for late or lost entries. Links that are not public at the deadline are not scored.
10. The latest information will be available on techfest.org, with registered participants notified of changes by email.
11. All technical, procedural and competition-related queries must be submitted

through the Techfest Competition Portal under the Doubt section. Questions about TrueForge itself are best asked in the TrueForge community Discord. Official announcements come from the email address shown on the competition page. Queries sent by email will not be answered.

12. Plagiarism, copying another team's submission, or sharing a submission between teams leads to disqualification of all teams involved.

Certificate policy

Teams shortlisted for Round 2 who take part on campus receive a Certificate of Participation from Techfest, IIT Bombay.

Prize money

Prize money is awarded to the winning teams by NEFT and processed by Jan 2027. Winners complete the prize money form on the Techfest website after Round 2 concludes; the form is visible through the registered email ID and must be submitted by 31 December 2026. Keep ready: account holder's name, account number, bank and branch name, and IFSC code.

Pre-Placement Interviews. Top-performing participants will be considered for Pre-Placement Interviews (PPIs) for internships at TrueFoundry. Internships secured through PPIs are eligible for consideration for full-time roles.

Award	**Prize**
1st place	INR 1,00,000
2nd place	INR 75,000
3rd place	INR 50,000
Social Media Award, first	INR 30,000
Social Media Award, two runners-up	INR 10,000 each
Social Media Award, five special mentions	INR 5,000 each

Resources

- **TrueForge GitHub repository: github.com/truefoundry/trueforge** · Docs · Quickstart · SDK · Discord
- VS Code Extension API: code.visualstudio.com/api
- Pinned repository: github.com/pallets/flask, tag 3.1.3
- Open-source editor-side references: Cline, Continue, Roo Code, opencode
- TrueFoundry: truefoundry.com · Agent harness docs